

Towards partial fulfillment for Undergraduate Degree Level Programme

Bachelor of Technology in Computer Science and Engineering

A Project Report on: DGvGAN, A Comparison of DGCNNs and GANs for Dynamic Malware Analysis

Prepared by:

Admission Number	Student Name
U22CS102	Ameya Srivastava
U22CS127	Auritra Dey
U22CS130	Bittu Kumar
U22CS121	Pavitra Kumar

Semester: VII

Year: 2025–2026

Supervised by: (Prof.) Dr. Devesh C. Jinwala



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

S. V. NATIONAL INSTITUTE OF TECHNOLOGY (NIT)

SURAT - 395007 (GUJARAT, INDIA)

Student Declaration

This is to certify that the work described in this project report entitled **DGvGAN, A Comparison of DGCNNs and GANs for Dynamic Malware Analysis** has been actually carried out and implemented by our project team consisting of the following members. Neither the source code therein, nor the content of the project report have been copied or downloaded from any other source. We understand that our result grades would be revoked if later it is found to be so.

Sr. No.	Admission No.	Student Name	Signature of the Student
1	U22CS102	Ameya Srivastava	
2	U22CS127	Auritra Dey	
3	U22CS130	Bittu Kumar	
4	U22CS121	Pavitra Kumar	

Certificate

This is to certify that the project report entitled **DGvGAN, A Comparison of DGC-NNs and GANs for Dynamic Malware Analysis** is prepared and presented by:

Sr. No.	Admission No.	Student Name
1	U22CS102	Ameya Srivastava
2	U22CS127	Auritra Dey
3	U22CS130	Bittu Kumar
4	U22CS121	Pavitra Kumar

students of B. Tech. Computer Science and Engineering and their work is satisfactory.

Signatures:

Supervisor

Jury

Head of Department

Abstract

Modern malware frequently employs obfuscation techniques that undermine traditional static analysis. Existent malware detection methods such as converting binary executables into grayscale images for image classification using CNNs are computationally expensive in deployment and risk information loss, misclassifications. They could also be easily evaded by polymorphic malware variants. Dynamic analysis, in contrast, captures the behavior of the program through API call sequences. Our project aims to implement two deep learning paradigms for this use-case. The first is a Deep Graph Convolutional Neural Network (DGCNN), following Oliveira and Sassi (2019)^[1], which models API calls as behavioural graphs to learn inter-call dependencies. The second reframes detection as anomaly identification via a Generative Adversarial Network (GAN) trained solely on benign samples, thereby distinguishing malicious activity as deviations from normal behaviour. We compare the supervised DGCNN with our GAN-based approach, evaluating their respective effectiveness against modern malware threats.

Table of Contents

1	Introduction	1
1.1	Motivation	1
1.2	Our Contribution	2
1.3	Outline of the Report	2
2	Related Work	3
2.1	Existing Research	3
2.2	Design Choices	4
3	Methodology	5
3.1	Dataset and Preprocessing	5
3.2	Approach 1: Supervised Detection with DGCNN	6
3.3	Approach 2: Anomaly Detection with a GAN	9
4	Results and Discussion	10
4.1	Model-1 (DGCNN) Results	10
4.1.1	Experiment 1, Balanced Dataset	10
4.1.2	Experiment 2, Imbalanced Dataset	11
4.1.3	Drawing Baseline Comparison	12
5	Conclusions and Future Directions	13
	Bibliography	14

Introduction

The proliferation of malware poses a persistent and evolving threat to software infrastructure worldwide. Modern malware authors frequently employ advanced evasion tactics such as **polymorphism** and **metamorphism**, which alter the malware’s code to create new variants. These obfuscation techniques ensure that no two instances have the same static signature, thereby rendering traditional signature-based detection (static) methods unreliable. The challenge requires a shift towards more intelligent and behaviour-oriented detection paradigms.

1.1 Motivation

The canonical method of converting malware binaries into grayscale images for analysis and detection via classification through a CNN is a static analysis technique. While it has been widely studied, it exhibits several critical shortcomings in practical deployment:

- **High computational cost** from reading, reshaping and preprocessing large binaries into images introduces significant latency.
- Flattening code into gray pixel values **discards key runtime behaviours** and control-flow relationships, leading to crucial information loss.
- Minor changes to the binary structure can drastically alter the image. Additionally, **static images do not capture** the program’s actual runtime behaviour.
- The need for intermediate images and large, deep CNNs **increases memory and processing requirements**, making it impractical in enterprise.

Another issue lies in the **common practice of downsampling binaries to fixed image sizes**. For example, a 10 MB binary contains 10×2^{20} bytes, while a standard 256×256 image holds only 2^{16} pixels. This results in a data loss of

$$\text{Loss} = \frac{10 \times 2^{20} - 2^{16}}{10 \times 2^{20}} \approx \frac{159}{160} \equiv 99.375\%,$$

meaning **nearly all byte-level information is discarded**. Using full-resolution images would preserve more detail, but are computationally infeasible to process at scale.

The aforementioned method of detection therefore trades speed, robustness and behavioural insight for visual intuition. These are the very limitations that **motivate dynamic, behaviour-focused approaches** such as **Deep Graph CNNs (DGCNNs)** or **GAN-based anomaly detection**, that directly model program execution and are more resilient to modern evasion tactics.

1.2 Our Contribution

The primary goal of this project is to implement and compare two distinct paradigms for dynamic malware detection:

1. To implement a supervised malware detector based on a **Deep Graph Convolutional Network (DGCNN)**, replicating the core methods from **Oliveira and Sassi (2019)**^[1] that models API call sequences as behavioural graphs. This implementation is used as **our baseline**.¹
2. To develop a second detector by reframing the task as an anomaly detection problem, using a **Generative Adversarial Network (GAN)** trained exclusively on benign goodware samples.
3. To conduct a comparative analysis of the two models, evaluating their performance, strengths and weaknesses in the context of dynamic malware analysis.

This dual evaluation aims to highlight the tradeoffs between supervised and unsupervised methods, acting as a benchmark for approaches that use generative models for detecting evolving or unseen malware.

1.3 Outline of the Report

Our report details the methods and approaches we took in our project. It will first cover the implementation of a Deep Graph Convolutional Neural Network (DGCNN) as our baseline for future experimentation. Then, in a later phase, we shall outline a second approach using a Generative Adversarial Network (GAN) for anomaly detection, where the model is trained only on benign samples to identify malicious behavior as deviations. Finally, we present a comparative analysis of the performance of these two distinct methods, drawing inferences and actionable insight from the same.

¹This mid-term evaluation report focuses on this first step.

Related Work

2.1 Existing Research

Deep Learning has introduced numerous powerful solutions for cybersecurity, with various network architectures being applied for both static and dynamic malware detection. While the classical deep learning models like Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) have been successfully applied, recent research has explored the use of Graph Neural Networks (GNNs) to capture more complex structural relationships in program behaviour.

Our project is primarily based on the foundational work by **Oliveira and Sassi (2019)**, in their paper “*Behavioural Malware Detection Using Deep Graph Convolutional Neural Networks*.” This was a pioneering study that to the best of the authors’ knowledge, was the first to investigate the direct application of Deep Graph Convolutional Neural Networks (DGCNNs) for behavioural malware detection using Windows API call sequences.

The core methodology proposed by the authors involves a novel two-step process:

1. **Graph Representation:** First, the temporal (time) sequence of API calls generated during a program’s sand-boxed execution is transformed into a spatial behavioural graph. In this directed graph, nodes represent unique API calls and an edge from one node to another signifies a direct temporal succession in the program execution trace.
2. **Graph Classification:** The behavioral graphs, along with its corresponding API call sequences are then fed into a modified DGCNN architecture. The model uses graph convolution to learn high-level feature representations that capture dynamic patterns in the program’s behavior.

To validate their approach, Oliveira and Sassi compared their DGCNN model against a Long Short-Term Memory (LSTM) network, which is a widely used architecture for sequence-based behavioral malware detection. Their results demonstrated that their implementation achieved comparable and often superior performance to the LSTM network, confirming API call graphs were a rich source of information for malware detection.

Our project extends a similar line of inquiry. We in turn adopt their DGCNN as our benchmark for a supervised learning approach. However, instead of comparing it to another sequence model like an LSTM, we introduce a fundamentally different method for comparison, a **Generative Adversarial Network (GAN)** for **anomaly detection** in benign samples. This allows us to explore how a supervised, graph-based method compares to a semi-supervised generative approach, particularly in real-world contexts.

2.2 Design Choices

Before we delve into the methodology, we find it necessary to justify some design choices we made for our project:

Why implement the DGCNN?

We implement the DGCNN because it is an established, novel methodology that performs dynamic analysis on malware by inspecting the API sequence calls (modeled as a behavioural graph) made in a sandbox environment. It serves as a modern and efficient alternative to the existing image translation based static binary classification technique through CNNs.

Why implement a GAN for anomaly detection? Why not simpler models?

Simpler models are faster and lighter, but they tend to fixate on maximizing class separation for classification rather than learning the true distribution of the classes involved. In our case, we aim to learn what truly separates malware from benign processes, aiming to model them as anomalous to normal behaviour. A GAN's **generator** can be forced to learn the complex relationships between API calls, so that it can produce realistic, benign and fake samples that could help the **discriminator** recognize anomalies accurately. The use of a GAN for the same purpose is yet to be explored in detail, giving us the opportunity to study and document it.

3

Methodology

In the first phase of our project, we have successfully implemented the supervised baseline detailed in the research paper^[1]. The second method to be integrated is a planned approach for the later phase that reframes the task as an anomaly detection problem using a Generative Adversarial Network (GAN).

3.1 Dataset and Preprocessing

Our work utilizes the public domain dataset published by the authors. It has documented **Windows API call sequences** from **42,797 malware** and **1,079 goodware** (benign) samples, making the data severely imbalanced. Per sample, the **top 100** from a total of **307** distinct calls are recorded. The original data is sourced from actual binaries executed in a Cuckoo Sandbox environment. As consulted from the research paper, the raw data underwent several preprocessing steps:

1. Each program’s behavior is represented by the **first 100 non-consecutive, repeated API calls** from its parent process.
2. A global vocabulary of all unique API calls was created, resulting in **307 distinct calls**. Each call was then mapped to a unique integer identifier (0 to 306).
3. We trained our baseline DGCNN model on both the **balanced** as well as **imbalanced** dataset, similar to **Experiments 1 & 2** outlined in the paper.

ID	API Call
0	NtOpenThread
1	ExitWindowsEx
2	FindResourceW
...	...
150	GetSystemInfo
151	SetFilePointer
152	Module32FirstW
...	...
306	__exception__

Table 3.1: A table depicting a sample of the 307 distinct API calls possible.

3.2 Approach 1: Supervised Detection with DGCNN

The data provided in the dataset was thoroughly transformed for the DGCNN implementation to be possible. Our data preprocessing steps included:

- Balancing the dataset to account for the class imbalance. We used **Random-Undersampling (RUS)** on the **majority (malware)** class, as outlined in the paper’s **Experiment 1**.
- Creating adjacency matrix-based graph (A) representations of each sample. Each sample recording the **top 100** API sequence calls based on the I.D. of each entry in the row is mapped to a 307×307 matrix that represents the structure of the behavioural graph (**Algorithm 1**).
- Populating feature matrices (X) to represent each sequence accurately. The input vector of 100 API call IDs is **one-hot encoded** into the adjacency matrix from the previous step into a shape of $(100, 307)$. This matrix is then **transposed** to the required shape of $(307, 100)$ to be compatible for matrix multiplication in the **graph convolution** operation (**Algorithm 2**).

Algorithm 1 API Sequence to Graph Adjacency Matrix (A)

```
1: Input: A list of API call sequences ‘sequences’
2:  $n \leftarrow \text{length}(\text{sequences})$ 
3:  $N \leftarrow 307$  ▷ Total number of unique API calls
4:  $A \leftarrow \text{zeros}(n, N, N)$ 
5: for  $i = 0 \rightarrow n - 1$  do
6:    $seq \leftarrow \text{sequences}[i]$ 
7:   for  $j = 0 \rightarrow \text{length}(seq) - 2$  do
8:      $u \leftarrow seq[j]$ 
9:      $v \leftarrow seq[j + 1]$ 
10:    if  $0 \leq u < N$  and  $0 \leq v < N$  then
11:       $A[i, u, v] \leftarrow 1$ 
12: return  $A$ 
```

Description: Algorithm 1 converts a list of API call sequences (rows from our data) into a 3D adjacency matrix A , where each slice $A[i]$ represents the directed connections between API calls in the i -th row sequence. Each entry in the matrix, $A[i, u, v] = 1$ indicates that API call u is immediately followed by API call v in that sequence. For the original dataset, this matrix takes a shape of **(43877,307,307)**, approximately **3.85GB**, since the data is stored as type **int8**.

Algorithm 2 API Sequence to Node Feature Matrix (X)

```
1: Input: A list of API call sequences ‘sequences’
2:  $n \leftarrow \text{length}(\text{sequences})$ 
3:  $N \leftarrow 307$  ▷ Total number of unique API calls
4:  $s \leftarrow 100$  ▷ Fixed length of each sequence
5:  $X \leftarrow \text{zeros}(n, N, s)$ 
6: for  $i = 0 \rightarrow n - 1$  do
7:    $seq \leftarrow \text{sequences}[i]$ 
8:   for  $j = 0 \rightarrow \text{length}(seq) - 1$  do
9:      $id \leftarrow seq[j]$ 
10:    if  $0 \leq id < N$  then
11:       $X[i, id, j] \leftarrow 1$ 
12: return  $X$ 
```

Description: Similar to the previous, Algorithm 2 converts a list of API call sequences into a 3D node feature matrix X , where each slice $X[i]$ corresponds to the i -th sequence. Each entry $X[i, u, j] = 1$ indicates that API call u occurs at position j in sequence i . The resulting tensor has shape **(43877, 307, 100)** for our dataset.

The two resultant tensors A and X from either algorithm are at the core of the model’s **Graph Convolution (Algorithm 3)** operation, which learns to aggregate information from a node’s neighbors according to the provided formula:

$$Z = f(\tilde{D}^{-1}\tilde{A}XW)$$

Where,

- X is the matrix of node features
- W is the weight matrix of the model itself
- $\tilde{A}$ is the augmented adjacency matrix $\tilde{A} = A + I$
- $\tilde{D}^{-1}$ is the normalization step, inverse of the degree matrix of $\tilde{A}$

The degree matrix $\tilde{D}$ is a diagonal matrix where each diagonal entry contains the number of neighbors for node i (and its self-loop). It is often multiplied in the convolution step as normalization because very large values (high degrees) can destabilize the network during training and cause bias. This is effective because it is equivalent to dividing each node’s aggregate vector by its degree, turning the sum into a normalized average. This keeps the scale of the feature vectors consistent across data.

The step, $\tilde{A}XW$ is the main aggregation. For each node, it effectively sums the transformed features XW of all its neighbors and itself.

The paper^[1] suggested we use the **row-normalized** version it presented, but it can disproportionately weigh high-degree nodes, causing them to dominate the aggregation. To address this, we adopt the most popular method for such models, using **symmetric normalization** instead:

$$Z = f \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} \cdot XW \right)$$

This is a direct result we consulted from **Kipf & Welling (2016)**^[2]. We chose to use this because by doing so we ensure the network balances contributions from both source and target nodes, reducing bias towards high-degree nodes. This is believed to improve generalization.

Algorithm 3 Graph Convolution Operation

- 1: **Input:** Adjacency Matrix A , Feature Matrix X , Weight Matrix W
 - 2: $I \leftarrow \text{Identity}(A.\text{shape})$ ▷ Create Identity matrix for self-loops
 - 3: $\tilde{A} \leftarrow A + I$ ▷ Augment Adjacency Matrix with self-loops
 - 4: $\tilde{D}_{ii} \leftarrow \sum_j \tilde{A}_{ij}$ ▷ Create diagonal degree matrix from row sums of $\tilde{A}$
 - 5: $\tilde{D}^{-1/2} \leftarrow \text{Diagonal}(\sqrt{\tilde{D}^{-1}})$ ▷ For symmetric normalization
 - 6: $\tilde{A}_{\text{norm}} \leftarrow \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$ ▷ Calculate normalized adjacency matrix
 - 7: $O \leftarrow \tilde{A}_{\text{norm}} XW$ ▷ Aggregate features and apply weight transformation
 - 8: $H \leftarrow \sigma(O)$ ▷ Apply activation function (e.g., ReLU)
 - 9: **return** H
-

Description: Algorithm 3 implements a single-layer Graph Convolutional Network (GCN). It performs symmetric normalization after augmentation, aggregating neighboring node features to apply a learnable weight matrix W . The result is then passed through an activation function σ to produce the updated node features H . This captures both local graph structure and node attributes for learning.

Our implementation for the DGCNN replicates **Model-1** from the paper, with the following architecture:

- A single **Graph Convolution Layer** based on the paper’s implementation
- A **ReLU (Rectified Linear-Unit)** activation function
- A **Dropout** layer with a rate of **0.60** for regularization
- A **Flatten** layer followed by **Dense** layers for final binary classification

The model was trained for at most 20 epochs, and early stopping was implemented to prevent any overfitting, since the model has a vast number of parameters. The forward propagation through the network layers is summarized in **Algorithm 4**.

Algorithm 4 Model Forward Propagation

- 1: **Input:** Adjacency Matrix A , Feature Matrix X
 - 2: $Z \leftarrow \text{GraphConv}(A, X, W_{\text{conv}})$ ▷ From Algorithm 3
 - 3: $H \leftarrow \text{ReLU}(Z)$ ▷ Apply activation
 - 4: $H_{\text{drop}} \leftarrow \text{Dropout}(H, \text{rate} = 0.60)$ ▷ Regularize
 - 5: $v \leftarrow \text{Flatten}(H_{\text{drop}})$ ▷ Flatten output to a single vector
 - 6: $v_{\text{dense}} \leftarrow \text{ReLU}(\text{Dense}(v, W_{\text{dense1}}))$ ▷ Hidden dense layer
 - 7: $O \leftarrow \sigma(\text{Dense}(v_{\text{dense}}, W_{\text{out}}))$ ▷ Sigmoid for binary probability
 - 8: **return** O
-

Description: Algorithm 4 outlines the forward pass of the model. Node features are first processed with the graph convolution layer, followed by ReLU activation and aggressive dropout for regularization. The resulting representation is flattened and passed through a hidden dense layer, finally mapped to an output vector with a sigmoid activation.

3.3 Approach 2: Anomaly Detection with a GAN

Our second planned approach (yet to implement) reframes malware detection as an anomaly detection problem to leverage the highly imbalanced nature of real-world malware data. Instead of learning from both malware and goodware, the Generative Adversarial Network will be trained **exclusively on benign (goodware) samples**.

- The **Generator (G)** will be tasked to learn the underlying distribution of normal API call sequences, producing realistic but synthetic goodware behaviours from random noise.
- The **Discriminator (D)** will be trained concurrently to become an expert at distinguishing between real goodware sequences from the dataset and the fake sequences produced by the Generator (G).

For inference, the G-model will be discarded. The D-model (the discriminator) will be then used as an anomaly detector. When a new sequence is presented, the discriminator will output a score. Sequences that conform to normal patterns (goodware) will receive a high score, while sequences that deviate significantly (malware) will receive a low score and be flagged as anomalous, mimicking binary classification.

Our ultimate aim is to train the **GAN** model as an anomaly detection task and compare its effectiveness for dynamic malware analysis against the baseline implementation we have established using **Model-1**.

Results and Discussion

4.1 Model-1 (DGCNN) Results

4.1.1 Experiment 1, Balanced Dataset

The original dataset included **42,797 malware samples** and **1,097 goodwill samples**, as aforementioned, making it very imbalanced. As detailed in the reference paper, **Experiment-1** was run on a balanced form of the data. To assess how well the model could learn the actual patterns when given an equal chance to see both classes, we performed **random-undersampling** on the majority class (malware) to match the distribution of the goodwill. An equal **50/50** balance was chosen due to the scarcity of goodwill samples. The model was hence trained on a total of **2,158** samples, with a **80/20** train-test split.

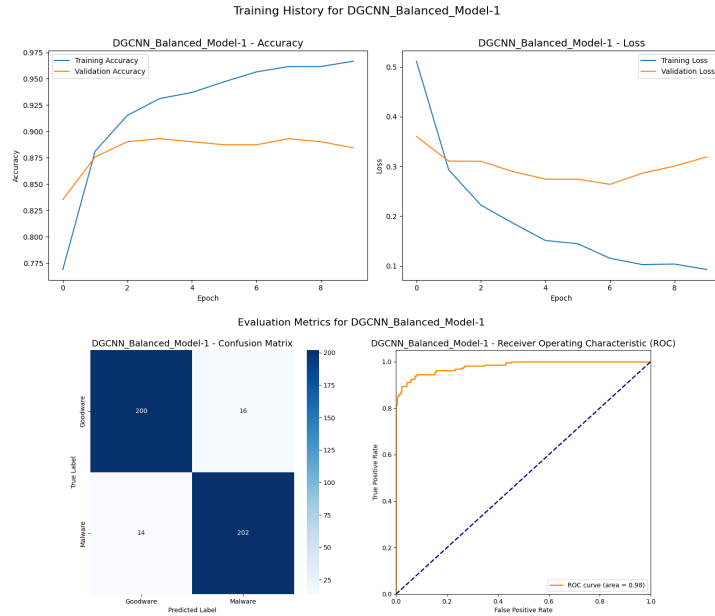


Figure 4.1: Training History & Model Evaluation Metrics for our implementation of Experiment 1. The model achieved a AUC-ROC of 0.98 with minimal misclassifications. Training was initially set for 20 epochs but early stopped at epoch 10 to prevent some mild overfitting behaviour, which begins around epoch 6. Despite the trend, recall remains high at 93.5% for goodwill and 92.5% for malware sequences, comparable to the paper authors’ implementation.

4.1.2 Experiment 2, Imbalanced Dataset

This implementation worked with the original data directly, since this naturally mimics deployment conditions, where real world data is imbalanced. The model’s architecture remained consistent in both experiments.

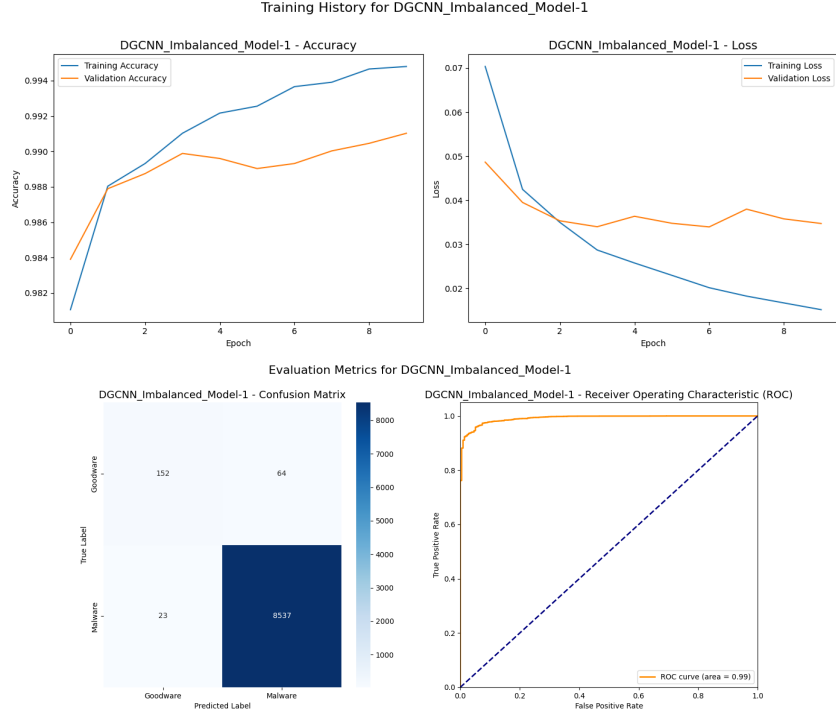


Figure 4.2: Training History & Model Evaluation Metrics for our implementation of Experiment 2. The model early stopped half-way of 20 epochs. Overfitting is evident due to severe class imbalance (40 malware : 1 goodware), but the calculated macro-average F1-score of 0.890 indicates satisfactory performance despite the skewed data.

The implementation of Model-1 in our reference paper^[1] was directly consulted. While the authors implemented a 2-layer convolution variation of the DGCNN as a separate Model-2 for comparison, their remarks suggested that there was no verifiable benefit of implementing the same, which is why we decided only implement the 1-layer convolution DGCNN (Model-1) itself.

4.1.3 Drawing Baseline Comparison

We compare our results for both experiments with the paper’s implementation below.

Table 4.1: Comparison of DGCNN Model Performance

Balanced Dataset (Experiment 1)		
Metric	Our Implementation	Paper’s Baseline
Accuracy	0.930	0.910
AUC	0.966 (best)	0.968 (best)
Precision	0.930	0.887
Recall	0.930	0.928
Imbalanced Dataset (Experiment 2)		
Metric	Our Implementation	Paper’s Baseline
AUC	0.983 (best)	0.968 (best)
F1-Score	0.890 (macro avg)	0.994*

*The paper reports an overall F1-score that is typically inflated in imbalanced datasets where the model correctly predicts the vast majority class. We believe that our macro-average F1-score of 0.89 is a more balanced measure of performance across both classes. We refrain from reporting accuracy since it is not a reliable metric to use here.

We conclude that we have successfully replicated **Model-1** from the reference paper^[1]. The model depicts acceptable performance in both **Experiments 1 & 2**. Further optimization of this model is **not** in the scope of our project as of present, since we treat the DGCNN as a mere baseline for future experiments.

Conclusions and Future Directions

We have successfully replicated the single-layer Dynamic Graph Convolutional Neural Network (DGCNN), or **Model-1**, from our reference paper by Oliveira and Sassi (2019), establishing a robust baseline for dynamic malware detection using API call sequences.

Our baseline implementation demonstrates strong performance on both balanced and imbalanced datasets, achieving an **AUC-ROC of 0.966** in the former and a **macro-average F1-score of 0.890** in the latter. The results indeed confirm that graph-based models are effective at capturing the complex, non-linear relationships inherent in program execution.

The DGCNN learns discriminative patterns that distinguish between malicious and benign software by modelling API call sequences as a behavioural graph. The validation of this model against the original paper’s results provides a reliable benchmark upon which our future implementation can be built.

With a solid DGCNN baseline established, our immediate future work will focus on the implementation and evaluation of an alternative approach using a **Generative Adversarial Network (GAN)** for anomaly detection.

Our planned next steps are as follows:

1. Train a GAN’s generator on benign API call sequences from our dataset to create realistic goodware samples, concurrently training the discriminator to learn to distinguish real from generated sequences.
2. Discard the generator and use the trained discriminator to identify anomalies, assigning low scores to malware that deviates from benign patterns.
3. Evaluate the GAN-based detector against the baseline we have implemented to test robustness and generalization in deployment.

This comparative study aims to provide valuable insights into the strengths and weaknesses of both methods studied.

Bibliography

- [1] A. L. I. Oliveira and R. J. Sassi, "Behavioral Malware Detection Using Deep Graph Convolutional Neural Networks," in *2019 International Joint Conference on Neural Networks (IJCNN)*, 2019. Available: https://www.researchgate.net/publication/336989176_Behavioral_Malware_Detection_Using_Deep_Graph_Convolutional_Neural_Networks
- [2] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," *arXiv preprint arXiv:1609.02907*, 2016. Available: <https://arxiv.org/abs/1609.02907>